California House Price Prediction — Project Report

Nov 6, 2026 · @Nirmay Kadam

Abstract

A tuned XGBoost model predicts the median house value of California census block groups with a test R^2 of 0.860 (RMSE $42,736, MAE $28,219), 37% lower error than least-squares Linear Regression (R^2 0.647). Using the 1990 California Housing dataset (20,640 block groups), the project covers data quality, missing values, encoding, EDA, outliers, feature engineering, least-squares optimisation, cross-validated model comparison, residuals and predictor interpretation. All models under-predict the 4.7% of blocks capped at $500,001, and R^2 falls to about 0.71 for unseen regions. The model is deployed in a Streamlit app (R^2 0.827), and the ethical risks of location-based valuation are discussed.

1. Introduction and objectives

Problem. Given the census attributes of a California block group, predict its median house value in US dollars. This is supervised regression on independent observations, evaluated with MAE, RMSE and R^2 on data the model has never seen.

Objectives. Assess and prepare a public dataset with evidence-based treatments; explore it; engineer features; fit and verify a least-squares Linear Regression; compare it with Random Forest and XGBoost under cross-validation; diagnose errors and interpret predictors; deploy the model; and discuss limitations and ethics.

2. Dataset and data acquisition

The data is the California Housing dataset: 20,640 rows and 10 columns from the 1990 U.S. Census. It was compiled by Pace and Barry (1997) and distributed through StatLib; the version used, which adds `ocean_proximity` and removes 207 `total_bedrooms` values, comes from Géron's *Hands-On Machine Learning* and is mirrored on Kaggle (`camnugent/california-housing-prices`).

Each row is a **census block group** of roughly 600–3,000 people, not a single house: counts are block totals and the target is the block's median value.

Variables: `longitude`, `latitude` (degrees); `housing_median_age` (years); block totals `total_rooms`, `total_bedrooms`, `population`, `households`; `median_income` ($10k); `ocean_proximity` (5 classes); target `median_house_value` (USD).

3. Data-quality assessment

The data is complete and consistent except for 207 missing bedroom counts; its main problem is **censoring** by the original collectors.

Check	Finding
Missing values	`total_bedrooms` only: 207 rows (1.0%)
Duplicates, impossible values	none
Censored target	**965 rows (4.7%) at exactly $500,001**
Censored features	age capped at 52 (1,273 rows); income at 15.0001 (49 rows)
Rare category	`ISLAND` has 5 rows; other classes 2,290–9,136
Skewed counts	skewness 3.4–4.9 for rooms, bedrooms, population, households

A model trained on this target cannot learn values above $500k, so expensive areas will be under-predicted (measured in Section 12).

4. Methodology and workflow

Split first. The data is split 80/20 (16,512 training and 4,128 test rows, `random_state=42`), stratified on five income bands because income is the strongest predictor. The split comes before any imputation, scaling or tuning, so nothing is learned from test rows. All tuning uses 5-fold cross-validation on the training set; the test set is used once, for the final scores.

One pipeline. Feature engineering, imputation, encoding and (for the linear model) log transform and scaling live in one scikit-learn `Pipeline`, defined in `housing_features.py` and saved with the model. The app passes raw values to that object, so it transforms inputs exactly as training did. This fixes a bug in the earlier version, where training log-transformed the counts but the app did not.

5. Missing-value treatment and categorical encoding

Missing values. Two strategies were compared with the same Linear Regression pipeline and 5-fold CV: dropping incomplete rows (16,354 rows) gave CV RMSE $67,088 and R^2 0.664; median imputation inside the pipeline (16,512 rows) gave $67,831 and 0.656.

The scores differ by about 1% because only 1% of rows are affected. Median imputation was chosen: it keeps every row, can score new properties with an unknown bedroom count, and is re-learned on each training fold, so it never sees validation or test data.

Categorical encoding. `ocean_proximity` has five unordered levels, so it is one-hot encoded rather than label-encoded (which would impose a false order). For Linear Regression the first level, `<1H OCEAN`, is dropped as the reference to avoid the dummy-variable trap; each coefficient then reads as the difference from `<1H OCEAN`. Tree models keep all five dummies. The earlier version dropped this column entirely, although it proves to be a top-four predictor.

6. Exploratory data analysis

Income is the strongest single predictor (r = 0.69), location acts in a strongly non-linear way, and the raw counts are skewed and collinear.

- **Univariate.** The count columns are strongly right-skewed (skewness 3.4–4.9; e.g. `total_rooms` mean 2,636 vs median 2,127), so the linear model gets a log transform. Income is moderately skewed (1.65); the target is right-skewed with a spike at the cap. Latitude is bimodal: Bay Area (~37.8°N) and Los Angeles (~34°N).
- **Bivariate.** Value rises with income but its spread widens (heteroscedasticity) up to a hard ceiling at $500k. `INLAND` blocks have a median near $108k against $215k–$235k for coastal classes. Expensive blocks cluster along the coast, a pattern no straight line in latitude and longitude can describe.

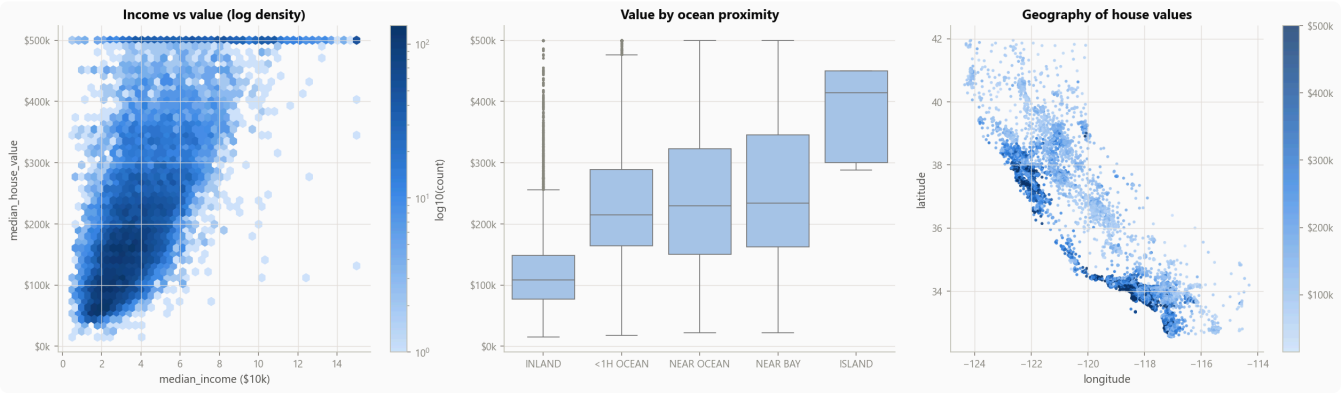


Figure 1. Value rises with income and hits a ceiling at $500k (left); inland blocks are far cheaper (centre); expensive blocks cluster on the coast (right).

- **Correlation.** Pearson r with value: `median_income` +0.688, `bedrooms_per_room` −0.256, `rooms_per_household` +0.152, `latitude` −0.144; the raw counts lie between −0.03 and +0.13. The counts correlate above 0.85 with each other, warning of multicollinearity (confirmed in Section 9). Spearman exceeds Pearson for income, so the relationship is monotonic but not linear.

7. Outlier identification

Two methods were compared. The **IQR rule** flags a row if any feature lies outside [Q1 − 1.5·IQR, Q3 + 1.5·IQR]. **Isolation Forest** isolates points that are easy to separate in the joint feature space; its contamination was set to the IQR flag rate so the two pick the same number of rows.

Each method flagged 3,938 rows (19.1%); 2,899 were flagged by both, a Jaccard overlap of 0.58.

IQR over-flags because the count columns are naturally heavy-tailed: most of its flags are simply large block groups. Isolation Forest also catches rows unusual in combination, such as many rooms per household with few residents (resort areas). **Decision: keep every row.** None is a data error, and removing them would delete the unusual neighbourhoods a model must handle. Their influence is controlled by the log transform (linear model) and by tree models, which split on ranks.

8. Feature engineering

Block totals mostly measure how big a block group is, not what its homes are like, so they were turned into per-household ratios computed from the raw counts:

Feature	Formula	Meaning
`rooms_per_household`	total_rooms / households	average home size
`bedrooms_per_room`	total_bedrooms / total_rooms	layout (higher = smaller homes)
`population_per_household`	population / households	crowding

The earlier notebook took logs before dividing, producing meaningless ratios of logs; this was corrected. The linear model additionally gets log(1 + x) on skewed columns and standardisation; trees need neither. The ratios pay off: crowding is the strongest engineered feature in both tree models, while each raw total adds under $1k to test RMSE once the ratios exist (Section 13).

9. Least-squares regression

Ordinary least squares chooses the coefficients β that minimise the residual sum of squares; setting the gradient to zero gives the normal equations:

$$\hat{\beta} = \arg\min_{\beta} \lVert y - X\beta \rVert^2 \quad\Rightarrow\quad X^\top X \, \hat{\beta} = X^\top y$$

Verification. Solving the normal equations by hand matched scikit-learn's `LinearRegression` to within 1.4×10^{-6}, and the SVD-based `np.linalg.lstsq` to within 2.6×10^{-9} (more accurate because it avoids forming X^TX).

Inference. A `statsmodels` fit gives $R^2 = 0.659$. Per standard deviation: income +$74.6k, latitude −$61.7k, longitude −$59.3k, age +$15.4k; an `INLAND` block is worth $31.5k less than a similar `<1H OCEAN` block. Nearly all terms are significant ($p < 0.05$), largely because $n \approx 16{,}500$.

Multicollinearity. VIF reaches 4,536 for `total_rooms` and 4,383 for `households` (vs 2.7 for income and 1.2 for age); after the log transform each ratio is a linear function of the counts. The symptoms are huge opposite-signed coefficients (`households` +$537k, `total_rooms` −$144k) and `bedrooms_per_room` turning positive despite its negative correlation. Predictions are unaffected, but only the low-VIF terms can be interpreted.

10. Models and hyper-parameter tuning

Five models were compared: a **baseline** that predicts the training mean; **Linear Regression**; **Linear Regression on log(value)**, to test whether a log target fixes the skew; **Random Forest** (bagged deep trees, CPU); and **XGBoost** (gradient-boosted trees with shrinkage and regularisation, trained on the GPU).

The tree models were tuned with `RandomizedSearchCV` over the full pipeline, scored by 5-fold CV RMSE:

- **Random Forest** (20 settings, 100 fits): best = 500 trees, max depth 30, max features 0.5, min leaf 1; CV RMSE $49,153.
- **XGBoost** (30 settings, 150 fits): best = 1,200 trees, depth 8, learning rate 0.03, subsample 0.7, column sample 0.6, L2 penalty 5; CV RMSE $45,053. Many small steps plus strong subsampling is the classic recipe for boosting that generalises.

GPU. XGBoost used `device="cuda"` on the RTX 3050, with automatic CPU fallback. On this small dataset one fit took 5.8 s on the GPU versus 3.6 s on the CPU, because data transfer outweighs the speed-up at 16,500 rows; the GPU still ran the 150-fit search in about 5 minutes without occupying the CPU. Random Forest stayed on the CPU, as scikit-learn has no GPU backend.

11. Validation and model comparison

XGBoost is best on every metric. **MAE** is the average error in dollars; **RMSE** weighs large errors more; **R^2** is the share of variance explained. All models used the same five folds, then were scored once on the test set.

Model	CV RMSE (mean ± std)	Test MAE	Test RMSE	Test R^2	Train R^2
Baseline (mean)	$115,699 ± 1,068	$90,807	$114,165	0.000	0.000
Linear Regression	$67,831 ± 742	$50,542	$67,807	0.647	0.659
Linear Regression, log target	$66,027 ± 972	$44,974	$79,670	0.513	0.678
Random Forest	$49,153 ± 877	$31,111	$46,826	0.832	0.976
XGBoost	**$45,053 ± 964**	**$28,219**	**$42,736**	**0.860**	0.983

Why the trees win. Value curves with income and location acts as coastal clusters, which a linear model cannot represent; trees also capture income × location interactions automatically. The log target lowers Linear Regression's MAE to $45.0k but worsens its test RMSE to $79.7k, because exp() inflates a few extreme predictions, so the problem is the functional form, not the scale. Linear Regression underfits (train–test R^2 gap 0.012); the trees overfit somewhat (gaps 0.144 and 0.123) yet generalise far better.

Random Forest vs XGBoost. XGBoost wins on all five folds by about $4,000 RMSE each (paired t-test $t = 23.5$, $p < 0.001$; indicative, as folds overlap).

Spatial validation. Neighbouring blocks are near-duplicates, so random folds partly test interpolation. GroupKFold on 0.5° grid cells (169 groups) holds out whole regions:

Model	Random K-fold R^2	Spatial R^2	RMSE increase
Linear Regression	0.656	0.604	+3%
Random Forest	0.819	0.716	+21%
XGBoost	0.848	0.710	+33%

For unseen regions, R^2 of about 0.71 is the honest expectation and XGBoost's lead over Random Forest disappears, though both still clearly beat Linear Regression.

12. Residual analysis

A residual is actual minus predicted value; positive means under-prediction.

- **Linear Regression:** residuals fan out as predictions grow (heteroscedasticity) and bend systematically (wrong functional form). It predicts **negative prices for 54 of 4,128** test blocks.
- **XGBoost:** residuals scatter around zero with no trend and no negative predictions. The Q-Q plot shows heavy tails, so large errors are more frequent than a normal distribution implies.
- **Location:** a residual map puts the largest XGBoost errors in the Bay Area and Los Angeles, where prices vary sharply and many targets are capped; inland errors are small.

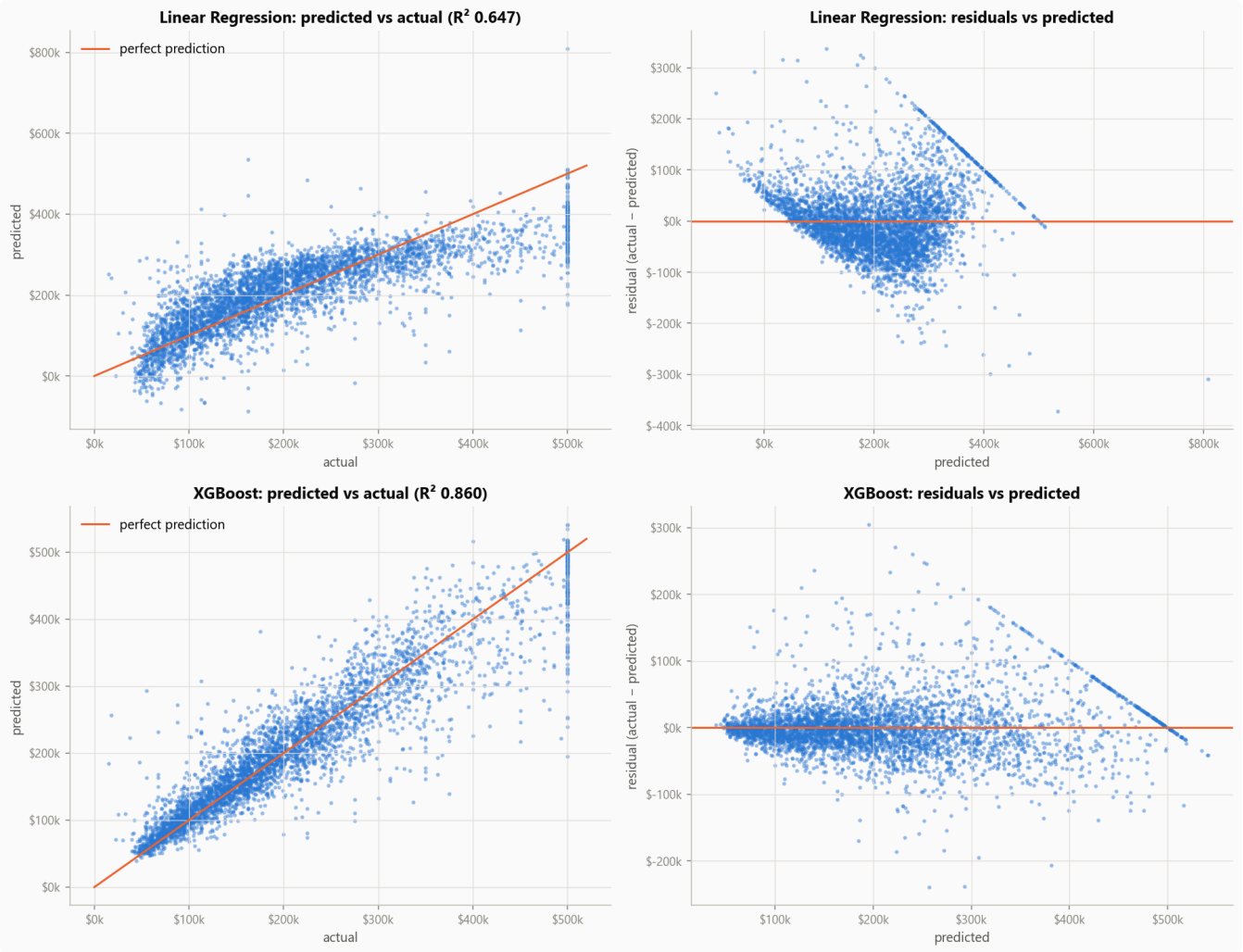


Figure 2. Top: Linear Regression shows a funnel and a curve. Bottom: XGBoost residuals scatter around zero. Both show the diagonal band of capped blocks.

The censored target. On the 179 test blocks capped at $500,001, XGBoost's MAE more than doubles ($58,973 vs $26,825 on uncapped blocks) and it under-predicts by $55,127 on average; Linear Regression under-predicts them by $127,485. Because the recorded value is only a floor, these errors are partly unknowable, and the model should not be used for high-value areas.

13. Interpretation of important predictors

Importance was measured by **grouped permutation importance** on the test set: the rise in RMSE when one feature is shuffled. Shuffling happens after feature engineering, and the five `ocean_proximity` dummies are shuffled together.

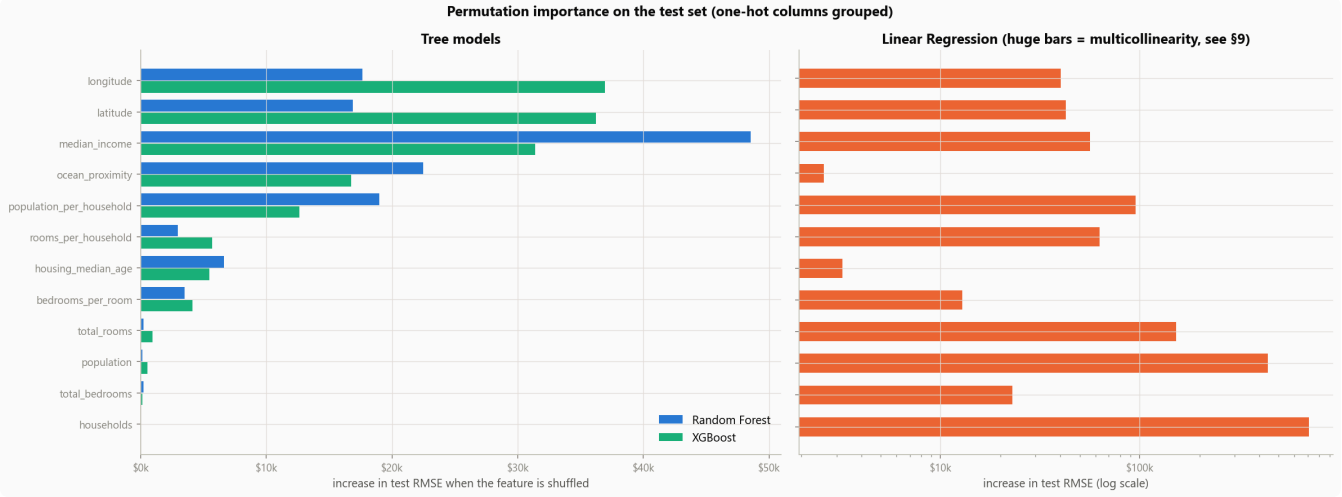


Figure 3. Grouped permutation importance on the test set. Linear Regression is on a log scale; its huge bars reflect multicollinearity.

- **Income and location dominate.** Random Forest ranks income first (+$48.5k); XGBoost spreads credit across longitude (+$36.9k), latitude (+$36.2k) and income (+$31.4k), using fine-grained location that overlaps with income.
- **Ocean proximity** adds +$17k to +$22k; by gain, `INLAND` is XGBoost's top split feature, and in OLS an inland block is worth $31.5k less.
- **Crowding** (`population_per_household`, +$13k to +$19k) is the strongest engineered feature; age adds a little.
- **Raw totals add under $1k each** once the ratios exist, justifying the feature engineering.
- **Linear Regression's bars are a warning:** shuffling `households` raises its RMSE by $708k because its collinear coefficients cancel each other (VIF ≈ 4,500). Its meaningful drivers are income (+$56k) and location (+$40k each).

14. Deployment: the Streamlit application

The best pipeline is deployed in a Streamlit map app. A user clicks a location and enters basic details; two helper models estimate what the user would not know.

- **Helpers** (trained on the same split): ocean proximity by k-NN (k = 3, test accuracy 0.992); median income by Random Forest, chosen by 5-fold CV over k-NN and XGBoost (test R^2 0.561).

- **Location check:** clicks more than 0.25° from any known block are rejected, which blocks the ocean, Nevada and Mexico.
- **Honest accuracy:** re-scoring the test set with the helpers' guesses in place of the true values gives R^2 0.827 and MAE $31,410 (vs 0.860 and $28,219). The loss is small because the model already sees location; the app shows this error beside every estimate.

15. Limitations and ethical issues

Limitations. 1990 prices; block-group medians, not houses; values above $500,001 hidden by censoring; R^2 about 0.71 for unseen regions and 0.83 in the app; and no data on schools, crime, transport or house condition.

Ethical issues in location and property data.

- **Proxy discrimination (digital redlining):** segregation and historical redlining tied location to race in the US, so a model that prices by location and income can reproduce discrimination with no protected attribute in the data.
- **Feedback loops:** if valuations drive lending, under-valued areas get less credit, prices stay low, and the next model confirms the under-valuation.
- **Privacy:** census data is aggregated to block groups so people cannot be identified; joining location and income with other records can undo that.
- **Uneven reliability:** errors are largest, and biased downwards, in expensive coastal and capped areas; a single average accuracy hides who bears the error.
- **Transparency and appropriate use:** the project reports what drives predictions and shows the app's error beside each estimate. The model suits teaching and exploration, not mortgage, insurance, tax or rental decisions about individuals.

16. Dashboard summary and conclusions

Decisions. Use a tree ensemble, preferring XGBoost within known areas (it and Random Forest tie on unseen regions); expect lower accuracy for expensive coastal blocks, distant regions and the app; and never use the model for decisions about individuals.

Conclusion. XGBoost cuts RMSE by 37% against Linear Regression. Censoring is the dataset's defining weakness, engineered ratios replaced raw totals as carriers of signal, and spatial cross-validation exposed an optimism of about 0.14 in R^2 that standard validation misses.

Future work. Model the censored target directly, add spatial features, report prediction intervals, and retrain on recent data.

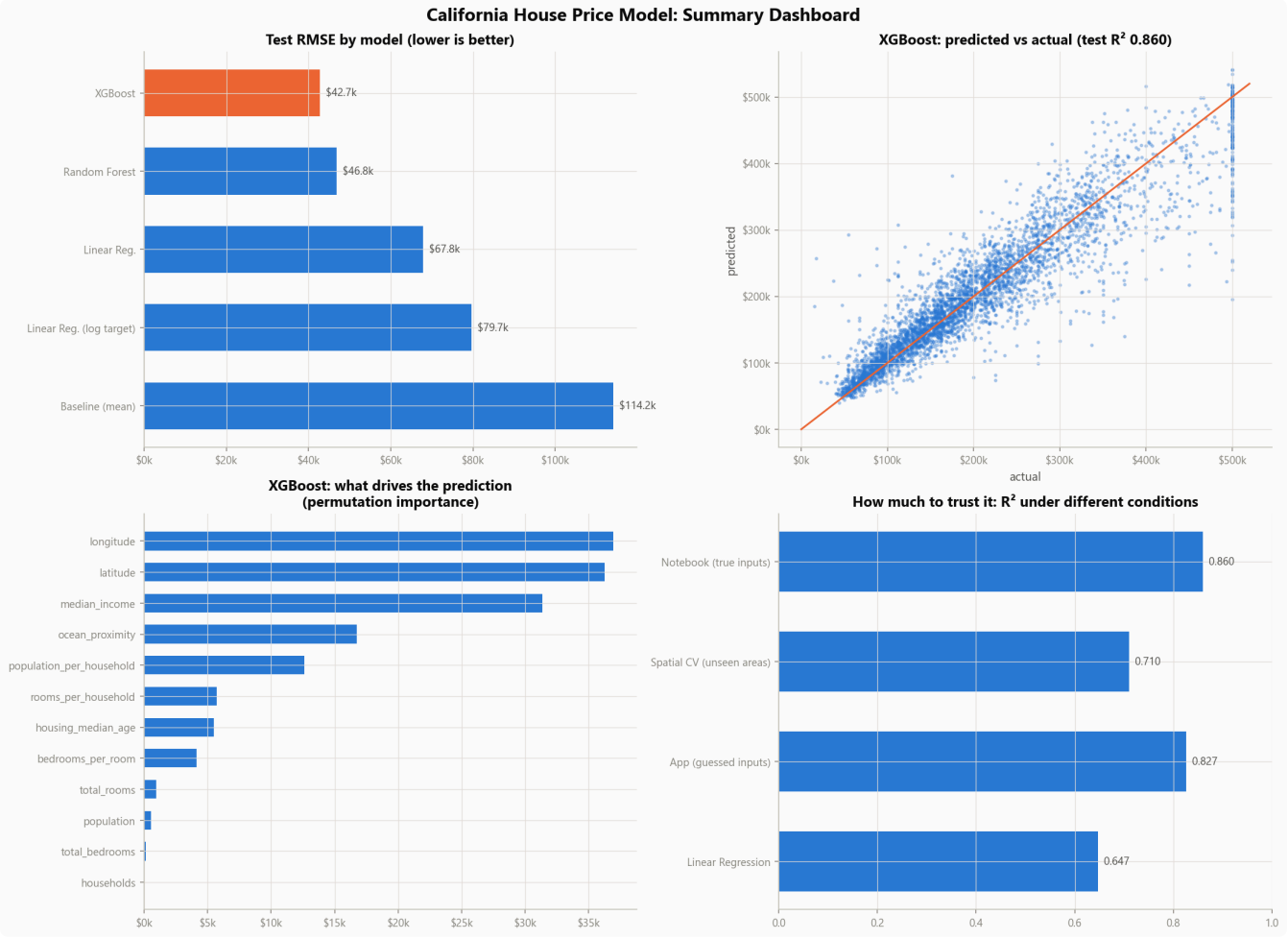


Figure 4. Summary dashboard: test RMSE by model, XGBoost predicted against actual, what drives its predictions, and R^2 under different conditions.

References

1. Pace, R. K. and Barry, R. (1997). Sparse Spatial Autoregressions. *Statistics & Probability Letters*, 33(3), 291–297.
2. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed. O'Reilly.
3. Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5–32.
4. Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proc. 22nd ACM SIGKDD*, 785–794.
5. Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). Isolation Forest. *Proc. 8th IEEE ICDM*, 413–422.
6. Roberts, D. R. et al. (2017). Cross-validation strategies for data with spatial structure. *Ecography*, 40(8), 913–929.